

M. Rahman

CPSC-1160

ID-100409251

Part 1-

Q-1:

- a. Output: ABCD
- b. Output: BBCD
- c. Output: Error
- d. Output: BCD
- e. Output: ABZD

BZD

Q-2:

- A. The problem in the code is memory leak. When the code uses `p = new int(20);` it creates an integer in the heap and the pointer is pointing to this one. But there was another value before but this line points the pointer to the new one and the older values address is lost.
- B. The memory problem is dangling pointer. As the pointer was deleted from the heap so, from the code the pointer tries to read from memory that is already deleted.
- C. The problem is wrong delete type. As the code tries to delete an array so using `delete[]` is appropriate here.
- D. Memory leak is the problem here. After the function finishes the pointer which held the key to the heap is gone and now the heap is still in the system but it is not accessible or I can't delete it anymore.
- E. Dangling pointer is the issue here. Here, p and q both pointed to the same memory in the heap but then the heap memory is deleted. So, this will cause a dangling pointer problem.
- F. This has no memory problem. The memory is deleted, and p is set to `nullptr` to avoid a dangling pointer.

Q-3:

- A. A dangling reference/pointer is more dangerous because it can immediately crash the program or change or read wrong memory. A memory leak is also bad, but usually it slowly wastes memory over time.

- B. A program can work with a memory leak and it can work perfectly. Memory leak mainly wastes memory and overtime it may cause the program to slow down or crash.